*https://github.com/cs-ubbcluj-ro/lab-work-computer-science-2024-tibumanu/tree/main/1-Mini-Language-And-Scanner/lab3*
Țibu Cristian
937/2

# Scanner (lexical analyzer) documentation

**Overview:**

The Scanner class is designed to perform lexical analysis on a given input file. It processes tokens, identifies reserved words, identifiers, and constants, and generates a Program Internal Form (PIF) and a Symbol Table (ST).

**Files:**

`Scanner.h`: Header file containing the declaration of the Scanner class and its methods.
`Scanner.cpp`: Implementation file for the Scanner class.
`main.cpp`: Contains the main function to execute the lexical analysis.
`PIF.out`: The Program Internal Form built by the scanner. Has entries of the form "`token positionInST`"
`ST.out`: The Symbol Table built by the scanner. Contains both the tree of identifiers and the tree of constants. Has entries of the form "`key index`".

**Methods:**

**`Constructor: Scanner()`**
Initializes the symbol table and reads reserved words from token.in; standard error *`"Error opening file token.in"`* when failing to open file.

**`void print()`**
Prints the identifiers and constants stored in the symbol table; calls print methods from the ST class.

**`void printReservedWords()`**
Prints the reserved words.

**`bool isIdentifier(std::string word)`**
Checks if the given word is an identifier, following the rules established in previous labs.

**`bool isConstant(std::string word)`**
Checks if the given word is a constant, following the rules established in previous labs.

**`void processToken(std::string &token, int line, std::ofstream &pif, SymbolTable &st)`**
Processes a token and adds it to the PIF and symbol table if necessary. An integer representing the current line is provided, alongside pointers for the currently opened files representing the output PIF and ST respectively; standard error *`"Lexical error at line <LINE>: <TOKEN>"`* if encountered a lexical error.

**`void writeSymbolTable(SymbolTable &st, std::ofstream &stf)`**
Writes the symbol table to a file - both trees in the form of "`key index`". Pointers are provided as arguments for the currently opened files representing the output PIF and ST respectively

**`lexicalAnalysis(std::string filename)`**
Performs the lexical analysis on a given file. The file should exist in the working directory. Tries to open the given `filename`, `PIF.out` and `ST.out`; standard error *`"Error opening file <FILE>"`* if any fail. The algorithm processes the input file character by character; starts building `` `currentToken` ``; when finding a white space, a newline or a separator, tries processing the current token. Separators and strings are handled separately.

**`main.cpp`**
This file contains the main function, which initializes the Scanner and performs the lexical analysis. Changes the work directory to `C:\uni\3\FLCD\lab3`. Lines 9-13 can be commented out if the user wishes to work with the default directory. User is asked to input the name of the file they wish to scan (e.g.: `p1.txt`)